



Community-based vulnerability prediction framework for IoT intrusion detection using only network topology

Fouad Al Tfaily ^{a,b,*}, Zakariya Ghalmane ^a, Mohamed El Amine Brahmia ^a,
Hussein Hazimeh ^{b,c}, Ali Jaber ^b, Mourad Zghal ^a

^a CESI LINEACT UR 7527, Strasbourg, France

^b Faculty of Sciences, Lebanese University, Beirut, Lebanon

^c Basic & Applied Research Center, Al Maaref University, Beirut, Lebanon

ARTICLE INFO

Keywords:

Internet of things
Community detection
Vulnerability prediction
Intrusion detection
Centrality metrics
Network topology

ABSTRACT

Internet of Things networks in critical infrastructure face sophisticated attacks that exploit structural vulnerabilities. Existing intrusion detection methods analyze individual devices, missing patterns that emerge when functionally related devices organize into communities that communicate more frequently with each other than with the rest of the network. Current structural analysis approaches examine either individual devices or entire networks, missing the intermediate community scale where attacks concentrate. Community-level analysis offers three advantages: capturing coordinated attack patterns, reducing computational complexity through grouping, and enabling proactive detection before attacks spread beyond community boundaries. Additionally, existing methods use fixed vulnerability criteria that fail across different network topologies. This paper proposes AdapCVP (Adaptive Community Vulnerability Prediction), a framework addressing these limitations through multi-scale structural analysis. AdapCVP detects communities using multiple algorithms, extracts features describing intra-community organization and inter-community relationships, then automatically determines vulnerability thresholds through cross-validation optimization. By operating exclusively on the network topology without examining flow content, AdapCVP preserves privacy and remains applicable across different classifiers. Experiments on three datasets compare 27 structural features capturing network topology against 28 data-flow features encapsulating traffic content. Structural features consistently outperform data-flow ones by 12.3 to 16.7% across machine learning, deep learning, and graph neural network classifiers. Vulnerability threshold optimization automatically discovers appropriate criteria ranging from 0.5% for distributed networks to 2% for centralized, and 3% for clustered, validating that vulnerability stems from structure rather than traffic volume and enabling deployment across heterogeneous IoT architectures without manual tuning. Moreover, ablation analysis demonstrates that the dominant structural drivers vary with network topology, reflecting differences in hub concentration, community clustering, and inter-community connectivity.

1. Introduction

The integration of Internet of Things (IoT) devices into critical infrastructure, from smart healthcare systems and industrial control networks to intelligent transportation and connected home environments, has created unprecedented security challenges that traditional intrusion detection systems struggle to address effectively. Modern IoT networks exhibit extreme diversity in their structure, ranging from centralized hub-dominated networks in smart home deployments to distributed configurations in large-scale sensor networks. This structural diversity de-

mands security approaches that can adapt to different network organizations without requiring manual parameter adjustment or deep knowledge of attack signatures.

Current intrusion detection research focuses primarily on analyzing individual devices, evaluating each one for suspicious behavior or known attack patterns [1,2]. However, IoT devices naturally organize into communities, which are groups of devices that communicate more frequently with each other than with the rest of the network [3]. For example, temperature sensors in a building zone form one community, while security cameras in another area form a different community.

* Corresponding author.

E-mail addresses: faltfaily@cesi.fr, fouad.tfaily@st.ul.edu.lb (F. Al Tfaily), zghalmane@cesi.fr (Z. Ghalmane), abrahmia@cesi.fr (M.E.A. Brahmia), hussein.hazimeh@ul.edu.lb (H. Hazimeh), ali.jaber@ul.edu.lb (A. Jaber), mzghal@cesi.fr (M. Zghal).

<https://doi.org/10.1016/j.future.2026.108493>

Received 12 December 2025; Received in revised form 16 February 2026; Accepted 24 March 2026

Available online 25 March 2026

0167-739X/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

These communities emerge based on functional roles, physical proximity, or shared purposes. Importantly, cyber attacks often target or spread through these community structures rather than isolated devices [4]. Despite this observation, existing approaches analyze devices individually, missing the collective vulnerability patterns that emerge at the community level. Community-based analysis offers three critical advantages over device-level approaches. First, it captures coordinated attack patterns that manifest across multiple devices simultaneously rather than treating each infection as an isolated incident. Second, it reduces computational complexity by grouping thousands of devices into manageable communities, enabling scalable security monitoring in large IoT deployments. Third, it enables proactive vulnerability detection before attacks spread beyond community boundaries, allowing targeted mitigation that contains threats within affected groups.

Network structure, which refers to the pattern of connections between devices, contains valuable information about security risks that can be extracted without examining private communication content [5–10]. Recent advances in graph-based IoT security have demonstrated the effectiveness of community-level structural analysis. Al Tjaily et al. [11] developed FedGATSage, a federated learning architecture combining Graph Attention Networks and GraphSAGE that analyzes structural patterns within IoT network communities to detect intrusions, achieving state-of-the-art performance while preserving data privacy through distributed training across multiple network sites. This work established that centrality measures, which are mathematical indicators of how important or central a device is in the network, can identify critical points whose compromise would disrupt network operation [12]. For instance, betweenness centrality measures how often a device lies on the shortest communication paths between other devices, while closeness centrality measures how quickly a device can reach all others in the network. However, prior structural approaches face two critical limitations. First, they extract features at a single scale, either analyzing individual devices or analyzing the entire network as a whole, missing the intermediate community level where attack patterns concentrate. Second, they use fixed analysis parameters that perform well on one network type but fail on others. A threshold that works for a centralized smart home network generates excessive false alarms in a distributed industrial sensor network.

While recent graph-based approaches have advanced intrusion detection through federated learning architectures that leverage community structure for privacy-preserving attack classification [11], these methods operate reactively on flow-level attack labels rather than proactively assessing which communities exhibit structural vulnerability before attacks occur. To address this gap in proactive community-level security assessment, this paper proposes AdapCVP, an Adaptive Community Vulnerability Prediction framework that predicts vulnerable IoT communities using only the topology of network connections while preserving privacy by discarding all packet content and traffic payloads. The framework begins by filtering raw network flow data to retain only structural identifiers, specifically which devices communicate with each other and when these communications occur, which are sufficient to construct a graph representation where devices become nodes and communication flows become edges. From this privacy-preserving graph structure, the approach applies community detection algorithms, including the Louvain method [13], greedy modularity optimization, and label propagation [14], to identify communities which are groups of densely connected devices. Once communities are detected, the framework extracts structural features that describe vulnerability patterns at two distinct levels of analysis. The first level examines the internal structure of each community, capturing how devices within the group connect to each other through measures such as internal density and the distribution of centrality values among community nodes. The second level examines how entire communities relate to each other in the broader network by treating each community as a single unit and analyzing properties such as whether a community serves as a communication bridge between other groups or remains isolated from the network core. Crit-

ically, AdapCVP introduces an adaptive thresholding mechanism that automatically adjusts the criteria for labeling a community as vulnerable based on observable network characteristics, specifically network density, which measures how many connections exist compared to how many are possible, and centralization, which measures whether connections concentrate around hub devices. This automatic adaptation ensures consistent performance across diverse IoT topologies without requiring manual tuning or attack signature databases.

Experimental evaluation on three structurally distinct IoT datasets demonstrates the effectiveness of the proposed approach, achieving F1-scores ranging from 0.846 to 0.948 across centralized, distributed, and clustered network topologies. The adaptive thresholding mechanism automatically selects appropriate vulnerability criteria, where the threshold determines what percentage of malicious traffic within a community triggers a vulnerability label, ranging from 0.5% for distributed networks where devices operate independently to 2% for centralized networks where hub devices naturally concentrate traffic, and up to 3% for clustered networks where dense local groups generate elevated baseline traffic. Comprehensive evaluation using multiple classifier families, including classical machine learning methods such as XGBoost and Random Forest, deep learning approaches such as Multi-Layer Perceptrons, and graph neural networks such as GraphSAGE, demonstrates that structural features consistently outperform data-flow traffic features across all classifier families, achieving F1-score improvements ranging from 12.3 to 16.7%, with an overall average improvement of 14.5%.

The remainder of this paper is organized as follows. Section 2 reviews related work in IoT security, community detection, and structural network analysis. Section 3 details the AdapCVP framework including graph construction from filtered structural identifiers, adaptive thresholding, and two-level feature extraction. Section 4 presents experimental validation on three diverse IoT datasets comparing performance with and without the proposed structural features. Section 5 concludes with implications and future research directions.

2. Related work

This section reviews prior research across three interconnected areas that inform our approach: intrusion detection systems for IoT networks, community detection algorithms that enable group-level analysis, and structural network analysis for security assessment. Throughout this review, we identify critical gaps that motivate the development of AdapCVP.

2.1. IoT security and intrusion detection systems

The security landscape of Internet of Things networks presents unique challenges that distinguish it from traditional enterprise environments, primarily due to resource constraints, protocol heterogeneity, and massive scale [15]. Early intrusion detection systems relied on signature-based methods that identified known attack patterns through pattern matching. While computationally efficient, these approaches demonstrated severe limitations when confronted with zero-day exploits and adaptive attackers who could modify their techniques to evade known signatures [15].

The limitations of signature-based systems motivated a shift toward machine learning approaches that could identify anomalous behavior without requiring explicit attack signatures. Classical methods, including Random Forests [16] and Support Vector Machines, demonstrated substantial improvements in detection accuracy while maintaining computational efficiency suitable for resource-constrained IoT devices [17]. Recent advances in deep learning have further improved capabilities through hierarchical feature learning, with convolutional and recurrent neural networks achieving state-of-the-art performance on benchmark datasets [18–20]. Autoencoder architectures have demonstrated particular effectiveness for anomaly detection in IoT botnet scenarios [21].

Despite these advances, current approaches focus predominantly on per-device analysis, treating network connections as independent events and missing the structural dependencies that characterize coordinated attacks. Furthermore, most methods require deep packet inspection to extract rich feature sets, limiting their applicability in privacy-sensitive environments where only connection metadata should be analyzed.

2.2. Community detection and structural analysis

Community detection algorithms identify groups of nodes that exhibit stronger internal connectivity than external connections, revealing organizational structure within complex networks. The Louvain algorithm emerged as the de facto standard for large-scale community detection due to its exceptional computational efficiency and effectiveness across diverse network types [13]. The method employs a two-phase iterative process that achieves near-linear time complexity, enabling analysis of networks with millions of nodes. Label propagation represents an alternative approach where each node iteratively adopts the most common label among its neighbors [14], offering even lower computational cost but potentially less stable results.

However, prior community detection research focuses predominantly on social, biological, and transportation networks, with limited exploration of IoT-specific challenges. IoT networks exhibit extreme sparsity with densities often below 0.001, high device heterogeneity, and dynamic topology changes. Furthermore, existing work treats community detection as an end goal rather than an intermediate step toward security applications. While community detection and graph-based analysis have been successfully applied to diverse domains such as automating user profile matching across social networks [22,23], their adaptation to IoT security contexts remains underexplored, particularly for vulnerability assessment applications.

While traditional security approaches analyze traffic content and attack signatures, an alternative research direction leverages network structure to identify threats. Centrality measures quantify the importance of individual nodes within network topology, with research demonstrating that network robustness depends heavily on structural properties [4]. Betweenness centrality identifies devices that control many communication pathways, whose compromise would fragment the network. However, these centrality-based approaches operate exclusively at the individual node level, missing collective vulnerability patterns that emerge at the community scale.

Recent advances have further refined community detection methodologies by addressing the challenges of overlapping community structures and incorporating centrality-aware mechanisms. Cheng et al. [24] proposed a Centrality-Aware Collaborative Learning (CACL) framework that integrates symmetric non-negative matrix factorization with kernel regression for overlapping community detection. The CACL approach introduces a centrality penalty mechanism to manage nodes with multiple community memberships and leverages both first-order and second-order adjacency information to preserve network structure comprehensively. While this approach advances community detection capabilities, it focuses on identifying community structures rather than predicting which communities exhibit vulnerability to attacks based solely on network topology.

Graph Neural Networks have emerged as powerful tools for learning from network-structured data. Graph Convolutional Networks introduced the foundational architecture for semi-supervised node classification [25], while Node2vec extended this through random walk-based embeddings [26]. Recent work has applied graph neural networks specifically to IoT intrusion detection. Termos et al. demonstrated that combining complex network measures with machine learning classifiers improves detection accuracy [27,28]. Hu et al. developed graph embedding methods for early network intrusion detection [29], while Tran and Park introduced novel approaches to enhance graph neural network performance [30]. Despite these advances, current graph-based approaches typically focus on node-level predictions

and require substantial computational resources that limit their applicability in resource-constrained IoT deployments.

Addressing the node-level limitation, FedGATSage [11] integrates client-side Graph Attention Networks with server-side GraphSAGE through community abstraction for privacy-preserving intrusion detection in distributed IoT networks. The approach achieves 78.58–82.78% balanced accuracy on node-level attack classification while reducing communication overhead by 85% through community-based embeddings, where communities detected via the Louvain algorithm serve as privacy-preserving aggregation units that protect individual device identities during federated parameter exchange. However, FedGATSage addresses intrusion detection, the reactive identification of ongoing attacks through real-time traffic analysis, rather than vulnerability prediction, which aims to assess structural susceptibility before attacks manifest. The prediction target is node-level or flow-level attack classification (benign vs. attack types on network flows) on labeled intrusion detection datasets, rather than community-level vulnerability assessment based solely on structural topology.

Beyond intrusion detection, several works have explored community structure in IoT security contexts with different objectives. Community-based security mechanisms [31] leverage game-theoretic community formation where devices mutually aid each other in secure software management and update authentication, but these communities represent logical groupings for cooperative behavior rather than graph-detected topological structures, and the focus is software deployment security rather than quantitative vulnerability assessment. Similarly, access control frameworks [32] use social communities and trust relationships to detect Sybil attackers in Social IoT environments, achieving over 90% detection accuracy, but the task is node-level attack detection rather than community-level vulnerability prediction based on structural properties.

The distinction between intrusion detection and vulnerability prediction represents a fundamental difference in security paradigms: detection systems identify attacks as they occur through supervised learning on labeled traffic data, while vulnerability prediction assesses structural characteristics that predispose organizational groups to compromise before attacks occur, enabling proactive security hardening and resource allocation. To the best of our knowledge, no prior work has addressed community-level vulnerability prediction based solely on network topology, where graph-detected communities serve as the prediction target rather than as intermediate representations for other tasks. Existing approaches consistently operate at node level (device-specific risk or attack classification), flow level (traffic-based intrusion detection), path level (attack graph analysis), or whole-network level (overall risk assessment), but not at the granularity of topological communities identified through structural analysis.

2.3. Adaptive methods and research gaps

The dynamic nature of IoT networks necessitates adaptive security mechanisms that can adjust to changing conditions without manual intervention. Recent developments in adaptive threshold mechanisms demonstrate that security systems can enhance detection performance by automatically adjusting sensitivity parameters [33]. However, most adaptive methods focus on temporal adaptation to traffic volume fluctuations rather than structural adaptation to topological diversity. They cannot automatically account for fundamental differences between centralized and distributed networks.

The development of comprehensive IoT datasets has been crucial for evaluating adaptive approaches across diverse network conditions [34–37]. Despite the availability of these diverse datasets, most adaptive security methods evaluate performance on individual datasets rather than demonstrating transferability across structurally distinct network types. Furthermore, to our knowledge, no prior work has developed a framework for community-level vulnerability prediction that operates exclusively on structural topology while automatically adapting

Table 1
Comparison of AdapCVP with Existing Approaches.

Approach	Analysis Level	Content Required	Adaptive Threshold	Predictive Framework
Traditional IDS [15]	Device	Yes	No	No
ML-based IDS [17]	Device	Yes	No	Limited
GNN Methods [25]	Device	No	No	Yes
Centrality [4]	Device	No	No	No
Dynamic Threshold [33]	Device	Yes	Yes	Limited
AdapCVP (Ours)	Community	No	Yes	Yes

vulnerability criteria to diverse networks ranging from centralized to clustered networks.

Table 1 positions AdapCVP relative to representative prior work across key dimensions including analysis granularity, adaptability to network heterogeneity, reliance on traffic content, and prediction capability. The reviewed literature reveals several critical limitations. First, existing intrusion detection systems predominantly operate at the device level, analyzing individual nodes in isolation and missing the collective vulnerability patterns that emerge when devices organize into functional communities. Even graph neural network approaches that explicitly model network structure typically perform node-level predictions rather than assessing community-level security posture. Second, most structural analysis methods require either deep packet inspection or extensive attack signature databases, limiting their applicability in privacy-sensitive IoT environments. Third, current approaches use static analysis parameters that perform well on specific network topologies but degrade on networks with different structural characteristics.

AdapCVP addresses these limitations through three key innovations. The framework operates at the community level, analyzing groups of functionally related devices to capture collective vulnerability patterns. The approach relies exclusively on connection topology, requiring only information about which devices communicate, without examining packet contents. Most critically, AdapCVP introduces systematic adaptation to network heterogeneity through automatic threshold selection based on density and centralization measurements, ensuring consistent vulnerability prediction across diverse IoT deployment scenarios from centralized smart homes to distributed sensor networks.

3. Methodology

This section presents the AdapCVP framework for predicting vulnerable communities in IoT networks using structural network analysis. We begin with an overview of the complete system architecture, then detail each component including dataset preprocessing and graph construction, community detection and multi-level feature extraction, adaptive threshold optimization for vulnerability labeling, multi-model prediction, and deployment considerations for time-varying IoT networks. Throughout this section, we emphasize that the framework operates exclusively on network connection topology without examining packet contents, making it suitable for privacy-sensitive IoT deployments.

3.1. Overall system design

The AdapCVP framework transforms raw network flow data into community-level vulnerability predictions through a systematic pipeline that leverages structural properties at multiple scales. Fig. 1 illustrates the complete workflow, which begins by filtering network traffic to retain only structural identifiers such as which devices communicate and when these communications occur, explicitly discarding all packet payloads and protocol-specific content. From this privacy-preserving representation, the approach constructs a graph where devices become nodes and communication relationships become edges, then applies community detection algorithms to identify groups of densely connected devices. The framework extracts features at two complementary levels of analysis, examining both the internal organization of each community

and how communities relate to each other in the broader network. An adaptive thresholding mechanism automatically determines appropriate vulnerability criteria based on the specific structural characteristics of the network being analyzed, ensuring consistent performance across diverse IoT topologies from centralized smart home systems to distributed sensor networks. Algorithm 1 presents the complete AdapCVP pipeline, showing how these components integrate to produce vulnerability predictions. The pipeline operates on network topology snapshots, enabling periodic re-execution as network structure evolves over time.

Algorithm 1 AdapCVP Framework.

Require: Network flow dataset D

Ensure: Community vulnerability predictions

```

1: Data Preparation:
2:  $(D_{train}, D_{test}) \leftarrow \text{ChronologicalSplit}(D)$ 
3:  $G_t \leftarrow \text{ConstructGraph}(D_{train})$ 
4: Community Analysis:
5:  $C \leftarrow \text{DetectCommunities}(G_t)$ 
6:  $F_{intra} \leftarrow \text{ExtractIntraFeatures}(C, G_t)$ 
7:  $F_{inter} \leftarrow \text{ExtractInterFeatures}(C, G_t)$ 
8: Vulnerability Labeling:
9:  $\tau^* \leftarrow \text{OptimizeThreshold}(F_{intra}, F_{inter})$ 
10:  $Y_{train} \leftarrow \text{AssignLabels}(C, \tau^*)$ 
11: Model Training:
12:  $F_{combined} \leftarrow \text{Concatenate}(F_{intra}, F_{inter})$ 
13:  $F_{train} \leftarrow F_{combined} \setminus \{\text{vuln\_ratio}\}$ 
14: Train classifier on  $(F_{train}, Y_{train})$ 
15: Prediction on Test Data:
16:  $G_{t+1} \leftarrow \text{ConstructGraph}(D_{test})$ 
17:  $C_{test} \leftarrow \text{DetectCommunities}(G_{t+1})$ 
18:  $F_{test} \leftarrow \text{ExtractFeatures}(C_{test}, G_{t+1})$ 
19: return Predict( $F_{test}$ )

```

3.2. Dataset preprocessing and node-Level graph construction

This subsection describes how raw network flow data is transformed into a graph representation suitable for structural analysis. The process begins with data filtering to retain only information necessary for topological analysis, followed by graph construction that captures communication patterns between devices.

3.2.1. Dataset preprocessing

The framework begins by filtering the raw network flow dataset to retain only structural identifiers while discarding all content that would require deep packet inspection. Specifically, we extract four (optionally three) attributes from each network flow record: the source IP address (identifying the device initiating communication), the destination IP address (identifying the receiving device), the timestamp (if existing, indicating when the communication occurred), and the label of the attack. This filtering step explicitly removes packet payloads, protocol headers, port numbers, and all traffic statistics that would reveal communication content, ensuring that subsequent analysis operates exclusively on connection topology. For example, a communication flow from device 192.168.1.5 to device 192.168.1.10 at timestamp T becomes simply a recorded connection between two nodes, with no information about what data was transmitted or which application generated the traffic.

We then partition the filtered dataset into training and testing portions through chronological splitting, where the first 70% of flows ordered by timestamp become the training set and the remaining 30% become the test set. This chronological separation ensures realistic evaluation where the framework learns vulnerability patterns from historical network behavior and predicts vulnerabilities in future network states, mimicking real-world deployment scenarios where security systems must forecast threats based on past observations. The chronological

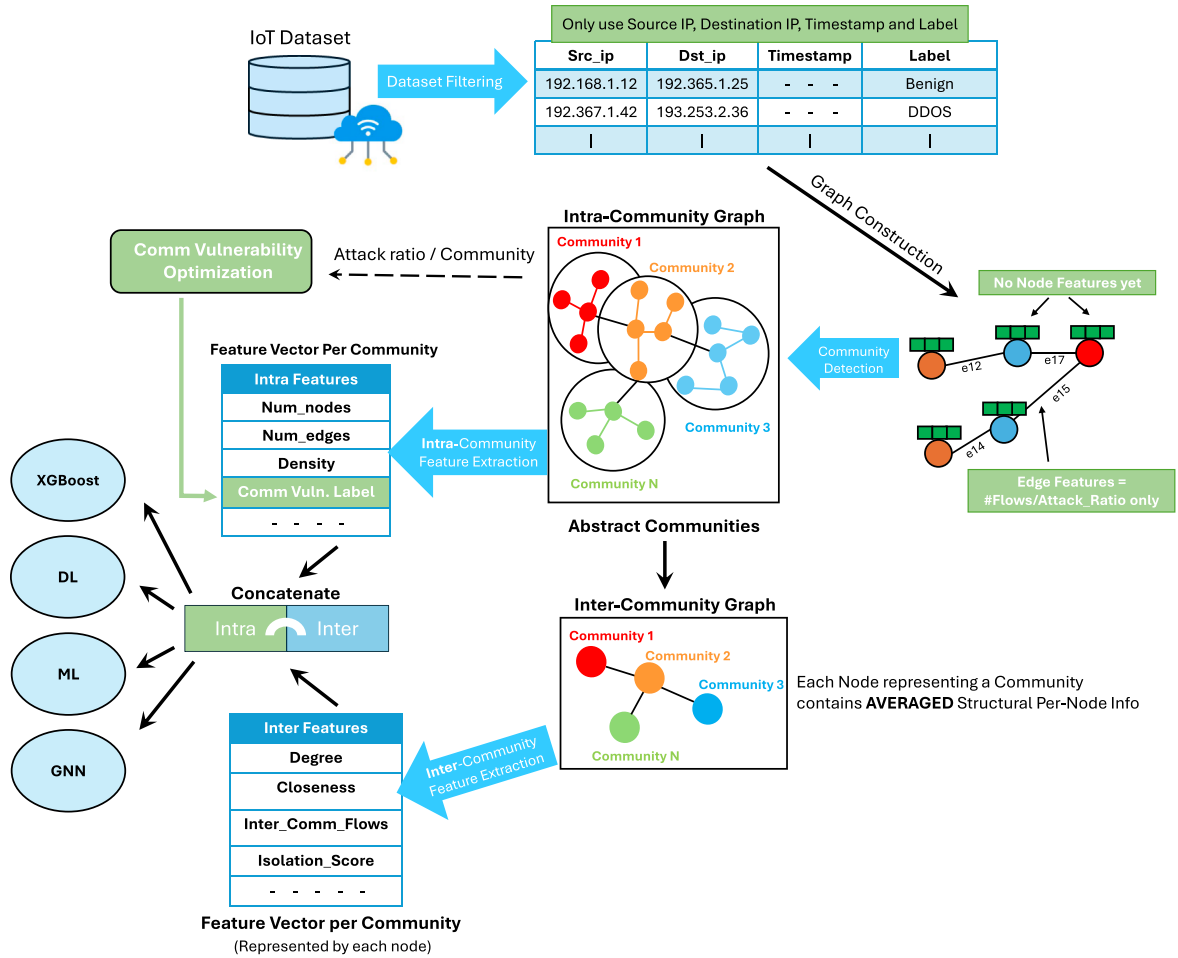


Fig. 1. The proposed AdapCVP architecture illustrating the four-stage pipeline. The process begins with Graph Construction from raw IoT datasets, followed by Community Detection where nodes are aggregated into functional groups. The core phase is Multi-Level Feature Extraction deriving intra-community and inter-community structural metrics. Finally, these features are concatenated in the Multi-Model Predictive Framework to detect vulnerability.

approach prevents information leakage that would occur if randomly sampled flows from throughout the observation period were used for both training and testing.

3.2.2. Node-Level graph construction

From the filtered structural identifiers, we construct an undirected graph $G_t = (V_t, E_t)$ representing the network topology at time t , where the vertex set V_t represents unique devices in the network and the edge set E_t represents communication relationships between devices.

Each unique IP address observed in the filtered dataset becomes a node in V_t , and an edge $(u, v) \in E_t$ exists if devices u and v exchanged at least one communication flow during the observation period. We weight each edge based on the volume of communication between its endpoints, specifically counting the total number of flows exchanged and the number of flows labeled as malicious by the original dataset annotations. These edge attributes, denoted as total flows and attack flows, capture communication intensity and threat concentration without revealing what specific information was transmitted.

This graph construction produces a network representation that preserves structural properties essential for vulnerability analysis while maintaining privacy by excluding all packet-level details. For instance, a small office IoT network with 50 devices might produce a graph with 50 nodes and 200 edges, where edge weights reveal that the central router communicates frequently with many devices while individual devices primarily connect to their local gateway. The resulting graph encodes

organizational structure, communication patterns, and threat distribution solely through connection topology.

3.3. Community detection and multi-Level feature extraction

This subsection presents the core analytical component of AdapCVP, which identifies communities within the network graph and extracts structural features at two distinct scales of analysis. We first describe the community detection process, then detail feature extraction at the intra-community level examining internal organization and the inter-community level examining relationships between groups.

3.3.1. Community detection algorithm

Operating on the network topology G_t constructed in Section 3.2.2, community detection partitions the graph into groups of nodes that exhibit stronger internal connectivity than external connectivity, revealing the natural organizational structure that emerges from device communication patterns.

We employ a multi-algorithm approach combining three complementary detection methods to ensure robust identification across diverse network topologies. The Louvain algorithm [13] serves as the primary method, employing greedy modularity optimization through iterative local moves and network aggregation to efficiently identify communities in large networks. We run the Louvain algorithm with multiple resolution parameters ranging from 0.3 to 3.0 to capture community structure at different granularities, where low resolution

values detect large coarse-grained communities while high values identify small fine-grained groups. Label propagation [14] provides a complementary perspective by iteratively assigning each node the label most common among its neighbors, naturally converging to community structure through local information exchange. Greedy modularity maximization offers a third detection strategy that incrementally merges communities to maximize the modularity quality metric.

The multi-algorithm approach produces multiple candidate community partitions, which we combine through a deduplication strategy that retains unique groupings while filtering redundant detections. For example, if Louvain with different resolution parameters produces overlapping partitions where 90% of nodes share identical community assignments, we retain only the distinct partition. This combination strategy ensures that subsequent analysis considers community structure at multiple meaningful scales rather than committing to a single potentially suboptimal partitioning. The result is a set of communities $C = \{C_1, C_2, \dots, C_n\}$ where each community $C_i \subseteq V$ represents a group of devices that communicate frequently with each other. In a smart building network, this process might identify distinct communities corresponding to HVAC sensors on each floor, security cameras in different zones, and lighting control systems in separate wings, reflecting the functional organization of the physical infrastructure.

3.3.2. Intra-Community graph feature extraction

The first level of structural analysis examines the internal organization of each community by constructing a subgraph containing only the devices and connections within that group. For each detected community $C_i \in C$, we define the intra-community subgraph $G_1^{(i)} = (V_1^{(i)}, E_1^{(i)})$ where the vertex set $V_1^{(i)} = C_i$ contains exactly the nodes belonging to the community and the edge set $E_1^{(i)} = \{(u, v) \in E \mid u, v \in C_i\}$ contains only edges connecting devices within the community. This isolation allows us to compute structural properties that describe how devices within the group relate to each other, independent of the broader network context.

From each intra-community subgraph, we extract features that capture three complementary aspects of internal organization: topological features describing connectivity structure, centrality-based features quantifying device importance, and communication volume features capturing aggregate traffic patterns. Topological features characterize the basic connectivity structure through metrics including the number of nodes and edges, network density, average clustering coefficient, transitivity, and Freeman's centralization index. Network density $\rho = \frac{2|E_1^{(i)}|}{|V_1^{(i)}|(|V_1^{(i)}|-1)}$ measures the proportion of actual connections to the maximum possible connections between community nodes [38]. The clustering coefficient $C = \frac{3 \times \text{number of triangles}}{\text{number of connected triples}}$ quantifies how often neighbors of a node are also connected to each other, where triangles are sets of three mutually connected devices and connected triples are sets with at least two edges [38]. Freeman's centralization index measures the extent to which connections concentrate around hub devices rather than being evenly distributed across the community [39]. For example, a tightly coordinated group of industrial control devices might exhibit high density and clustering, reflecting their intensive mutual communication, while a loosely connected set of independent sensors would show lower values across these metrics.

Beyond topological structure, centrality-based features quantify the importance and influence of individual devices within their community by computing centrality measures on the intra-community subgraph. Betweenness centrality measures how often a device lies on the shortest communication paths between other community nodes, identifying devices that control information flow within the group. Closeness centrality measures how quickly a device can reach all other community nodes, indicating devices that can rapidly disseminate information or malware throughout the group. Degree centrality counts how many direct connections each device maintains within its community. We com-

Table 2

Intra-Community Structural Features.

Feature	Description
<i>Topological Features</i>	
num_nodes	Number of devices in the community
num_edges	Number of connections within the community
density	Proportion of actual to maximum possible connections
avg_clustering	Average clustering coefficient across devices
transitivity	Global clustering coefficient
avg_degree	Average number of connections per device
max_degree	Maximum connections of any device
degree_variance	Variance in device connectivity distribution
edge_connectivity	Minimum edges needed to disconnect community
<i>Centrality Features</i>	
avg_betweenness	Average betweenness centrality across devices
max_betweenness	Maximum betweenness centrality
avg_closeness	Average closeness centrality across devices
max_closeness	Maximum closeness centrality
avg_degree_centrality	Average degree centrality across devices
<i>Communication Volume Features</i>	
total_flows	Total communication volume within community
attack_flows	Total malicious flows within community
avg_flows_per_edge	Average flows per connection
max_flows_per_edge	Maximum flows on any connection

pute these centrality measures for all devices in the community, then aggregate using statistical operators such as mean and maximum to produce community-level descriptors. A community might have high average betweenness centrality if most devices serve as communication intermediaries, or high maximum betweenness if a single device dominates as a hub while others remain peripheral.

Complementing structural properties, communication volume features capture aggregate traffic patterns within the community using the edge weight attributes from graph construction. We sum the total flows and attack flows across all edges in the intra-community subgraph to quantify overall communication volume and threat concentration within the group. We also compute the vulnerability ratio as the proportion of flows within the community that were labeled as malicious, which serves as a preliminary indicator of community security posture. For instance, a community with 10,000 total internal flows and 500 attack flows would have a vulnerability ratio of 0.05, suggesting that 5% of its internal traffic exhibits malicious characteristics. Table 2 summarizes the complete set of 18 intra-community features extracted from each community subgraph, organized by feature category.

3.3.3. Inter-Community abstraction feature extraction

The second level of structural analysis examines how entire communities relate to each other in the broader network by constructing an abstraction graph where each community becomes a single node. This macro-scale perspective reveals which communities serve as communication bridges connecting different network regions, which communities remain isolated at the network periphery, and which communities occupy central positions in the overall organizational structure.

We construct the inter-community abstraction graph $G_2 = (V_2, E_2)$ where the vertex set $V_2 = \{c_1, c_2, \dots, c_n\}$ contains one super-node for each detected community and the edge set E_2 contains edges between super-nodes based on inter-community communication volume. An edge $(c_i, c_j) \in E_2$ exists if devices in community C_i communicate with devices in community C_j with sufficient volume to indicate a meaningful relationship. We weight each edge by the total number of flows exchanged between the two communities, capturing the intensity of inter-group communication. For example, if community C_1 containing building management sensors exchanges 5000 flows with community C_2 containing HVAC controllers but only 50 flows with community C_3 containing security cameras, the abstraction graph would show a strong weighted edge between c_1 and c_2 but a weak edge to c_3 .

Table 3
Inter-Community Structural Features.

Feature	Description
<i>Structural Positioning Features</i>	
super_degree	Number of other communities connected to this community
super_weighted_degree	Total inter-community flow volume
super_avg_neighbor_weight	Average communication volume with neighboring communities
isolation_score	Degree of disconnection from network core
inter_community_flows	Total flows exchanged with other communities
<i>Centrality Features</i>	
super_betweenness	Betweenness centrality in abstraction graph
super_closeness	Closeness centrality in abstraction graph
super_clustering	Clustering coefficient in abstraction graph
hub_score	Eigenvector centrality indicating connection to important communities

From the abstraction graph, we extract features that capture each community's role in the broader network organization. Structural positioning features describe how communities connect to each other, including super-degree counting how many other communities each group communicates with, weighted degree summing the total inter-community flow volume, and isolation score measuring how disconnected a community is from the network core. A community with high super-degree participates in diverse inter-group communications and likely plays a coordinating role, while a community with low super-degree and high isolation score operates relatively independently at the network periphery.

Centrality-based features quantify each community's importance in the global network structure by computing centrality measures on the abstraction graph. Super-betweenness measures how often a community lies on the shortest paths between other communities in the abstraction graph, identifying groups that serve as critical communication bridges in the overall network. Super-closeness measures how quickly a community can reach all other communities through multi-hop paths in the abstraction graph, indicating groups that occupy central positions with efficient access to the entire network. Hub score based on eigenvector centrality identifies communities that connect to other highly connected communities, revealing hierarchical structure where some groups occupy more prominent organizational positions. For instance, a community of gateway devices bridging multiple building zones would exhibit high super-betweenness, reflecting its role as a necessary intermediary for cross-zone communication, making it a potentially high-value target for attackers seeking to disrupt network-wide operations. Table 3 summarizes the complete set of 9 inter-community features that characterize each community's role in the broader network organization.

3.4. Adaptive threshold optimization for vulnerability labeling

The framework requires labels indicating which communities are vulnerable to guide the training of predictive models. Rather than using fixed vulnerability criteria that perform well on some network types but poorly on others, AdapCVP employs an adaptive optimization procedure that automatically determines appropriate thresholds based on the specific structural characteristics of the network being analyzed.

The vulnerability labeling process begins with the vulnerability ratio computed during intra-community feature extraction, which measures what proportion of flows within each community were labeled as malicious. A community might have a vulnerability ratio of 0.02 indicating that 2% of its internal traffic exhibits attack characteristics, while an-

other community might have a ratio of 0.15 indicating 15% malicious traffic. Formally, we compute this ratio as $\theta_C = \frac{\text{malicious flows in } C}{\text{total flows in } C}$ where C represents a community. This community-level vulnerability ratio aggregates node-level attack labels, where a flow is considered malicious if either endpoint device was labeled as compromised in the original dataset annotations. The critical question is what threshold value should distinguish vulnerable communities from secure ones, particularly given that appropriate thresholds vary dramatically across network types. In a distributed network where devices operate independently, a threshold of 0.005 might be appropriate to flag communities with even minimal attack presence, while in a centralized network where hub devices naturally concentrate diverse traffic, a threshold of 0.02 might be necessary to distinguish genuine compromise from normal hub activity. In clustered networks where devices form dense local groups, higher thresholds around 0.03 may be required to avoid false alarms from elevated intra-cluster communication

The adaptive threshold optimization procedure, detailed in Algorithm 2, systematically searches for the threshold value that maximizes vulnerability prediction performance on the training data. We define a search space of candidate thresholds ranging from 0.005 to 0.3, evaluated over a fine-grained grid to ensure coverage of reasonable sensitivity levels from highly permissive to very strict. For each candidate threshold, we temporarily label communities as vulnerable if their vulnerability ratio exceeds that threshold value, creating a provisional set of labels. We then evaluate how well these labels enable vulnerability prediction by training a lightweight proxy classifier, specifically a Random Forest with 50 trees and maximum depth of 5 to maintain computational efficiency, using stratified 3-fold cross-validation on the structural features extracted in Sections 3.3.2 and 3.3.3. The threshold that produces the highest average F1-score across cross-validation folds becomes the selected vulnerability criterion for the graph in hand.

Algorithm 2 Adaptive Community Vulnerability Labeling Optimization.

Require: Communities C with features F_{intra}, F_{inter}
Ensure: Optimal threshold τ^* , vulnerability labels Y

- 1: Initialize: $\tau^* \leftarrow 0.05$, best_score $\leftarrow 0$
- 2: $F \leftarrow \text{Concatenate}(F_{intra}, F_{inter})$
- 3: Define threshold set $\mathcal{T}$ spanning [0.005, 0.300]
- 4: **for** $\tau \in \mathcal{T}$ **do**
- 5: $Y_{temp} \leftarrow \text{Label communities where } \text{vuln_ratio} > \tau$
- 6: **if** only one class in Y_{temp} **then**
- 7: **continue**
- 8: **end if**
- 9: scores $\leftarrow []$
- 10: **for** each fold in 3-Fold-Cross-Validation(F, Y_{temp}) **do**
- 11: Train Random Forest on training fold
- 12: Compute F1-score on validation fold
- 13: Append score to scores
- 14: **end for**
- 15: avg_score $\leftarrow \text{Mean}(\text{scores})$
- 16: **if** avg_score > best_score **then**
- 17: best_score $\leftarrow \text{avg_score}$
- 18: $\tau^* \leftarrow \tau$
- 19: **end if**
- 20: **end for**
- 21: $Y \leftarrow \text{Label communities using } \tau^*$
- 22: **return** τ^*, Y

This optimization procedure automatically adapts to network characteristics since structural properties influence which threshold values produce learnable vulnerability patterns. In networks where communities naturally exhibit diverse internal traffic characteristics, moderate thresholds around 0.02 to 0.05 typically perform best by distinguishing genuinely compromised communities from those with normal traffic diversity. In networks where most communities maintain consistently low

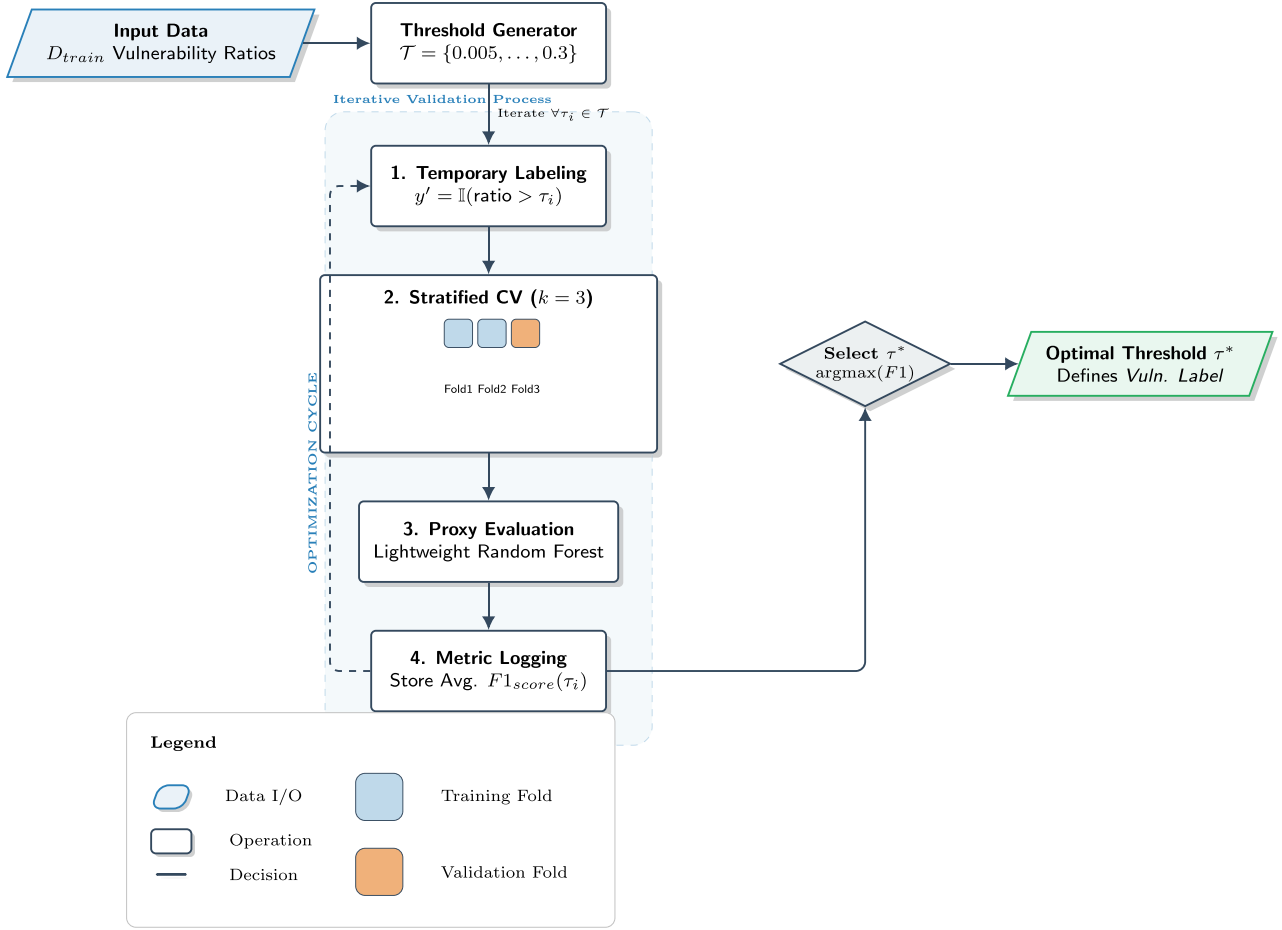


Fig. 2. The Adaptive Threshold Optimization cycle. The framework iterates through a candidate search space to generate temporary vulnerability labels based on attack ratios. A proxy classifier evaluates the separability of these labels using Stratified Cross-Validation. The threshold that maximizes the global F1-score is selected as the optimal definition of vulnerability.

attack ratios with only severely compromised groups showing elevated values, stricter thresholds below 0.01 may optimize prediction performance. The F1-score optimization criterion balances precision and recall, ensuring that selected thresholds neither produce excessive false positives by labeling too many communities as vulnerable nor excessive false negatives by missing genuinely compromised groups. Fig. 2 illustrates how the optimization process evaluates candidate thresholds to identify the value that maximizes cross-validation performance.

3.5. Feature concatenation and multi-Model prediction

The final stage integrates the extracted structural information and trains predictive models to forecast community vulnerability. We concatenate the intra-community features and inter-community features to form a unified structural representation for each community, combining local organizational properties with global positioning characteristics. This concatenation produces a feature vector with 27 dimensions per community, where 18 dimensions describe internal structure and 9 dimensions describe external relationships.

Critically, we remove the vulnerability ratio from the feature vector before training predictive models, even though this ratio was used to generate vulnerability labels through the adaptive threshold optimization procedure. Including the vulnerability ratio as a training feature would create circular reasoning where the model learns to simply threshold this single feature rather than discovering more complex structural vulnerability patterns. By excluding it, we force the models to learn how combinations of topological properties, centrality measures,

and communication patterns predict vulnerability, enabling generalization to networks where attack labels may not be available for threshold computation, therefore the resulting training feature matrix containing 27 structural descriptors per community, capturing organizational properties is derivable purely from the network topology without requiring knowledge of which specific flows are malicious.

We train multiple classifier families to demonstrate that vulnerability prediction performance stems from the quality of structural features rather than a specific algorithmic choice. Classical machine learning models including XGBoost gradient boosting and Random Forest ensemble methods provide strong baseline performance through effective handling of feature interactions and class imbalance. Deep learning approaches using Multi-Layer Perceptrons with multiple hidden layers demonstrate that neural network architectures can also leverage structural features effectively. Graph neural network methods using GraphSAGE specifically adapted to operate on the community-level abstraction graph show that graph-aware architectures provide additional benefits when structural relationships are explicitly encoded.

For fair comparison, we establish the baseline by training the same classifier families using data-flow traffic features that capture communication intensity through 28 metrics including forward/backward packet counts, byte rates, flow durations, protocol distributions, packet length statistics, and inter-arrival times. These data-flow features represent traditional approaches that analyze traffic volume and patterns without considering the organizational structure captured by our community-level framework. The performance difference between models trained on the complete 27-feature structural representation versus

the 28-feature data-flow baseline would quantify the contribution of our proposed framework, demonstrating whether structural topology provides advantages over traffic volume analysis for vulnerability prediction.

3.6. Deployment in time-Varying IoT networks

Internet of Things deployments experience continuous topology changes as devices join or leave networks, communication patterns evolve, and organizational structures are reconfigured. The methodology addresses these dynamics through snapshot-based execution, where each analysis operates on the current network state $G_t = (V_t, E_t)$ representing active devices and their communication relationships during the observation period. The complete pipeline (community detection, feature extraction, vulnerability prediction) executes independently on each topology snapshot without requiring historical network states or assuming structural stability across time periods.

When the framework is deployed in operational environments, the trained predictive model generalizes to newly discovered communities without requiring retraining. The model learns decision boundaries over the 27-dimensional structural feature space rather than memorizing specific community identities from training data. When re-execution on evolved topology G_{t+1} detects a previously unobserved community, the framework extracts the same structural features (centralization, density, inter-community connectivity) and classifies based on similarity to learned vulnerability patterns. The adaptive threshold mechanism recalibrates automatically for each network state based on structural characteristics, ensuring appropriate vulnerability criteria without manual adjustment. Re-execution frequency depends on network dynamics and operational requirements: highly dynamic environments may execute hourly assessments, stable infrastructures perform daily or weekly predictions, or event-driven triggering executes when topology changes exceed specified thresholds such as device churn rate or community structure similarity.

4. Experimental results

This section presents a comprehensive experimental evaluation of the AdapCVP framework across three benchmark IoT intrusion-detection datasets namely the NF-ToN-IoT, CIC-ToN-IoT, and IoT-23 datasets. All experiments use chronological 70/30 train-test splits to ensure realistic evaluation, enabling models to predict future vulnerabilities based on historical patterns. We organize the analysis as follows: First, we characterize the structural properties of the three datasets to establish their topological diversity. Second, we evaluate the effectiveness of the community-detection component across multiple algorithms. Third, we assess the performance of adaptive threshold optimization. Fourth, we compare vulnerability-prediction accuracy across classifier families and feature sets relative to baseline approaches. Fifth, we analyze the impact of incorporating node-level traffic statistics. Finally, we examine the computational complexity and runtime performance.

4.1. Dataset characteristics and structural diversity

Table 4 presents the structural characteristics of the three evaluation datasets, which exhibit fundamentally different network topologies. To contextualize these differences, we summarize the key graph metrics considered: network density measures the proportion of actual connections to maximum possible connections, centralization quantifies the extent to which connections concentrate around hub devices, and the clustering coefficient captures the prevalence of triangular connectivity patterns where neighbors of a device are also connected to each other [38]. The three datasets span a wide range of topological characteristics: NF-ToN-IoT exhibits the highest density (0.001608) and centralization (0.732) with relatively low clustering (0.039); IoT-23 demonstrates moderate centralization (0.544) with lower density (0.000358)

Table 4

Structural Characteristics of Evaluation Datasets.

Dataset	Nodes	Edges	Density	Centralization	Clustering	Classification
NF-ToN-IoT	1500	1808	0.001608	0.731856	0.039323	Centralized
IoT-23	6559	7699	0.000358	0.544334	0.000759	Distributed
CIC-ToN-IoT	49,339	230,419	0.000034	0.403544	0.246868	Clustered

Table 5

Community Detection Algorithm Comparison on NF-ToN-IoT Dataset.

Algorithm	Reference	Train Comms	Test Comms
Louvain (resolution = 1.0)	[13]	42	15
Label Propagation	[14]	38	13
Greedy Modularity	[41]	35	12
Girvan-Newman	[42]	29	11
AdapCVP Multi-Algorithm Proposed		57	19

and minimal clustering (0.0008); and CIC-ToN-IoT displays ultra-low density (0.000034) with the lowest centralization (0.404) but substantially higher clustering (0.247).

We classify networks based on their structural organization following established network science principles [38]. NF-ToN-IoT represents a centralized architecture where high centralization indicates hub devices dominate communication patterns, typical of smart home deployments with central routers coordinating peripheral sensors. IoT-23 exhibits a distributed structure with minimal local clustering (0.0008), characteristic of networks where devices operate independently without forming tight groups despite moderate centralization. CIC-ToN-IoT exemplifies a clustered configuration where devices form dense localized communication clusters (clustering 0.247) with minimal inter-cluster connectivity, despite overall low network density. This structural diversity enables rigorous evaluation of the framework's ability to adapt vulnerability detection criteria across fundamentally different network topologies ranging from centralized hub-dominated networks to clustered ones. Fig. 3 visualizes these structural differences, illustrating how NF-ToN-IoT's hub-dominated architecture contrasts with IoT-23's distributed structure and CIC-ToN-IoT's clustered structure.

4.2. Community detection analysis

Table 5 compares community detection outcomes across different algorithms on the NF-ToN-IoT dataset as an example. Our multi-algorithm approach combines Louvain with varying resolution parameters, label propagation, and greedy modularity optimization. After removing duplicate community structures, this produces 57 unique training communities and 19 unique test communities, representing a 36% increase compared to using any single algorithm. For comparison, Louvain alone detects 42 communities, label propagation finds 38 communities, and greedy modularity finds 35 communities.

Detecting communities at multiple resolutions proves critical for comprehensive vulnerability analysis [40]. IoT networks naturally organize into functional groups at different scales, from fine-grained clusters of sensors within individual rooms to coarse-grained zones spanning entire building floors. Single-algorithm detection captures only one organizational scale, potentially missing threats that manifest at other granularities. For instance, a targeted attack on room-level sensor clusters would be invisible if the detection algorithm only identifies floor-level zones, while a coordinated building-wide attack might be fragmented if detection focuses exclusively on room-level groups. By combining multiple detection methods, the framework identifies vulnerability patterns across all organizational scales simultaneously. The strategy of removing duplicate structures ensures that similar detections from different algorithm runs do not bias the analysis, while genuinely distinct perspectives from each method are preserved.

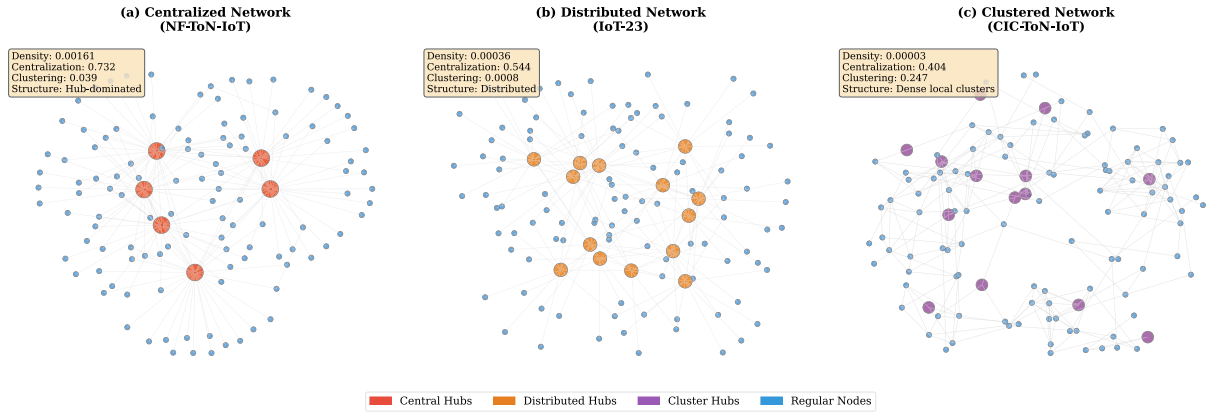


Fig. 3. Visual comparison of the three structurally distinct network topologies used for evaluation. (a) NF-ToN-IoT exhibits centralized hub-dominated architecture where peripheral devices connect primarily to central hubs, resulting in high centralization (0.732) and low clustering (0.039). (b) IoT-23 demonstrates distributed structure with devices spread independently and minimal clustering (0.0008), producing moderate centralization (0.544). (c) CIC-ToN-IoT displays clustered configuration where devices form densely connected local clusters with minimal inter-cluster connectivity, yielding low centralization (0.404) but high clustering (0.247).

Table 6
Comprehensive Threshold Sensitivity Analysis Across Datasets.

Threshold	NF-ToN-IoT (Centralized)			IoT-23 (Distributed)			CIC-ToN-IoT (Clustered)		
	F1	Vuln	Safe	F1	Vuln	Safe	F1	Vuln	Safe
0.005	0.8343	31	18	0.8872	30	5	0.6952	38	86
0.010	0.8343	31	18	0.8412	26	9	0.6952	38	86
0.020	0.8343	31	18	0.7548	21	14	0.7097	37	87
0.030	0.7656	29	20	0.7548	21	14	0.7178	36	88
0.040	0.7656	29	20	0.7477	20	15	0.6952	34	90
0.050	0.7656	29	20	0.7488	19	16	0.6952	34	90
0.100	0.7656	29	20	0.7488	19	16	0.6952	34	90
0.150	0.7656	29	20	0.6500	17	18	0.5117	30	94
0.200	0.7656	29	20	0.6287	15	20	0.5117	30	94
0.300	0.8276	25	24	0.4259	12	23	0.5249	29	95

4.3. Vulnerability threshold optimization

Table 6 presents comprehensive threshold sensitivity analysis across the three evaluation datasets, revealing dramatically different optimal vulnerability criteria. NF-ToN-IoT exhibits a performance plateau at low thresholds before declining at moderate values and partially recovering at high thresholds, with optimal selection at 0.020 to maintain peak performance while providing a safety buffer against noise. IoT-23 demonstrates monotonic degradation from optimal performance at the lowest threshold 0.005, with F1-scores progressively declining as thresholds increase and vulnerable community counts dropping from 30 to 12. CIC-ToN-IoT shows gradual improvement from low thresholds to peak performance at 0.030, before degrading at higher values. These distinct patterns reflect fundamental differences in how vulnerability manifests across network topologies, with centralized networks requiring moderate thresholds, distributed networks performing best at minimal thresholds, and clustered networks requiring the highest thresholds.

These threshold differences validate our topology-first design principle: vulnerability stems primarily from organizational structure rather than traffic volume. As illustrated in Fig. 4, the centralized NF-ToN-IoT network requires threshold 2% because hub communities naturally concentrate diverse traffic from many devices, requiring a moderate threshold to distinguish genuine systematic compromise from normal hub coordination activity. The distributed IoT-23 network with minimal local clustering achieves best performance at 0.5% because devices operate independently without forming tight groups, making any attack concentration highly significant, with performance degrading by 46 percentage points at higher thresholds that incorrectly dismiss coordinated threats as isolated noise. The clustered CIC-ToN-IoT network requires 3% be-

cause devices form dense local groups where elevated intra-cluster traffic is normal behavior rather than attack indication. A static threshold optimized for one topology type fails catastrophically on networks with opposite structural characteristics, degrading F1-scores by up to 40 percentage points and reducing detected vulnerable communities by up to 60 percent. The adaptive optimization procedure automatically discovers appropriate criteria for each network type, ensuring consistent performance across diverse IoT deployments without manual tuning.

4.4. Overall community vulnerability prediction

Table 7 presents a fair comparison of community vulnerability prediction performance using data-flow features versus structural features across multiple classifier families. Structural features consistently outperform data-flow features across all three datasets and four classifiers, with performance gains ranging from 12.3 to 16.7% depending on network topology. Tree-based methods (XGBoost, Random Forest) achieve the strongest absolute performance with structural features, reaching perfect F1-scores of 1.000 on IoT-23 and CIC-ToN-IoT. Neural network approaches (MLP, GraphSAGE) show substantial improvements with structural features, with GraphSAGE demonstrating particular effectiveness on CIC-ToN-IoT (0.933) where graph-aware architectures can exploit inter-community relationships. The overall average improvement of 14.5% across all datasets and classifiers demonstrates consistent superiority of structural features regardless of network topology or algorithmic choice.

This consistent superiority validates the fundamental premise that vulnerability stems from organizational structure rather than traffic volume. data-flow features capture communication intensity through metrics like packet counts and byte rates, indicating how much data devices exchange but providing limited information about how devices organize into functional groups or which devices occupy critical coordination roles. Structural features distinguish cases by capturing hub centrality, density patterns, and inter-community connectivity that reveal organizational vulnerabilities invisible to volume-based metrics. As shown in Figs. 5, 6, and 7, structural feature learning enables accurate vulnerability prediction by capturing organizational patterns that generalize across unseen communities. When encountering a previously unseen test community displaying comparable hub centrality, density, and attack distribution, the model correctly predicts vulnerability with 89% confidence based on learned structural patterns rather than simply thresholding individual metrics. The full AdapCVP feature set combining intra-community organization, inter-community positioning, and adaptive vulnerability labeling enables models to learn complex

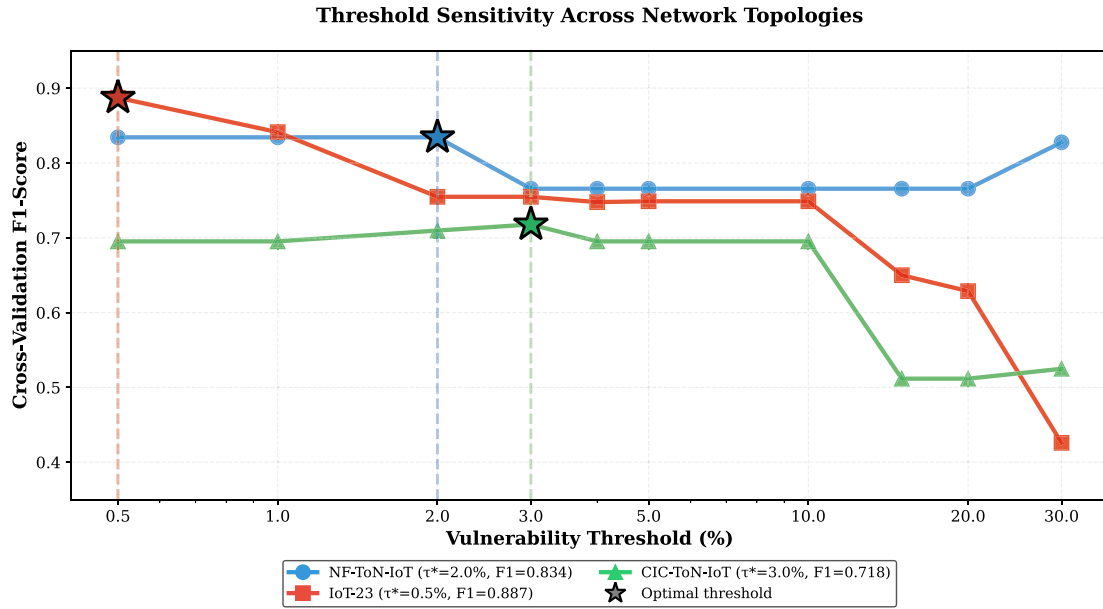


Fig. 4. Threshold sensitivity analysis demonstrating how optimal vulnerability thresholds vary dramatically across network topologies. NF-ToN-IoT with centralized hub structure achieves peak F1-score at 2% threshold. IoT-23 with distributed topology and minimal clustering requires the lowest threshold (0.5%) where any attack concentration signals vulnerability, with performance degrading monotonically by 46 percentage points at higher thresholds. CIC-ToN-IoT with clustered structure and dense local groups peaks at 3%. Static thresholds optimized for one topology type fail catastrophically on networks with opposite structural characteristics, validating the necessity of adaptive threshold selection.

Table 7

Data-flow versus Structural Feature Comparison for Vulnerability Prediction.

Dataset	Flow Features (28)					Structural Features (27)					Gain
	XGB	RF	MLP	GNN	Avg	XGB	RF	MLP	GNN	Avg	
NF-ToN-IoT	0.778	0.818	0.762	0.533	0.723	0.941	0.941	0.875	0.625	0.846	+ 12.3%
IoT-23	0.667	0.769	0.714	0.714	0.716	1.000	0.909	0.909	0.714	0.883	+ 16.7%
CIC-ToN-IoT	0.857	0.750	0.750	0.857	0.804	1.000	1.000	0.857	0.933	0.948	+ 14.4%
Overall Average Improvement:											+ 14.5%

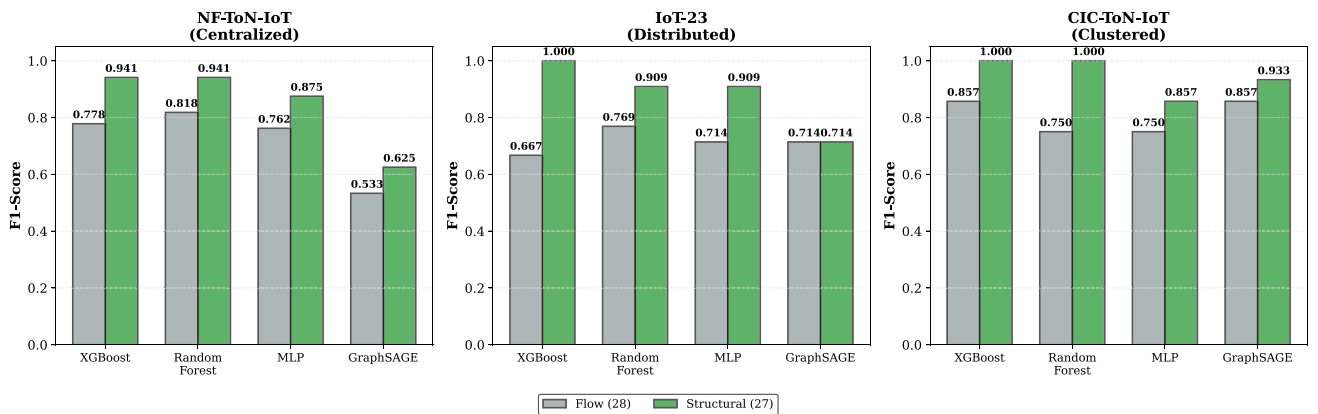


Fig. 5. F1-score comparison across classifiers and datasets, demonstrating consistent performance improvements when using AdapCVP's 27-dimensional structural feature set compared to 28 data-flow features. Structural features achieve superior performance across all classifiers, with tree-based methods (XGBoost, Random Forest) reaching perfect F1-scores of 1.000 on IoT-23 and CIC-ToN-IoT, and overall average improvements of 14.5% across all datasets and classifier families.

structural vulnerability patterns that generalize effectively across diverse network topologies.

4.5. Impact of data-Flow information integration

Table 8 evaluates whether incorporating aggregated data-flow traffic statistics into the community-level analysis provides additional predictive value beyond purely structural features. We augment the 27 structural features with five averaged data-flow metrics: mean packets per

flow, mean bytes per flow, mean flow duration, packet rate variance, and byte rate variance. Across all datasets and classifiers, incorporating these aggregates produces minimal performance changes, with F1-score differences ranging from -0.070 to +0.018 and average absolute differences of only 0.012 to 0.041 across datasets.

This marginal impact reinforces that community vulnerability stems primarily from organizational structure rather than traffic volume characteristics. Network topology encodes how devices are organized into functional groups, which devices occupy critical coordination roles,

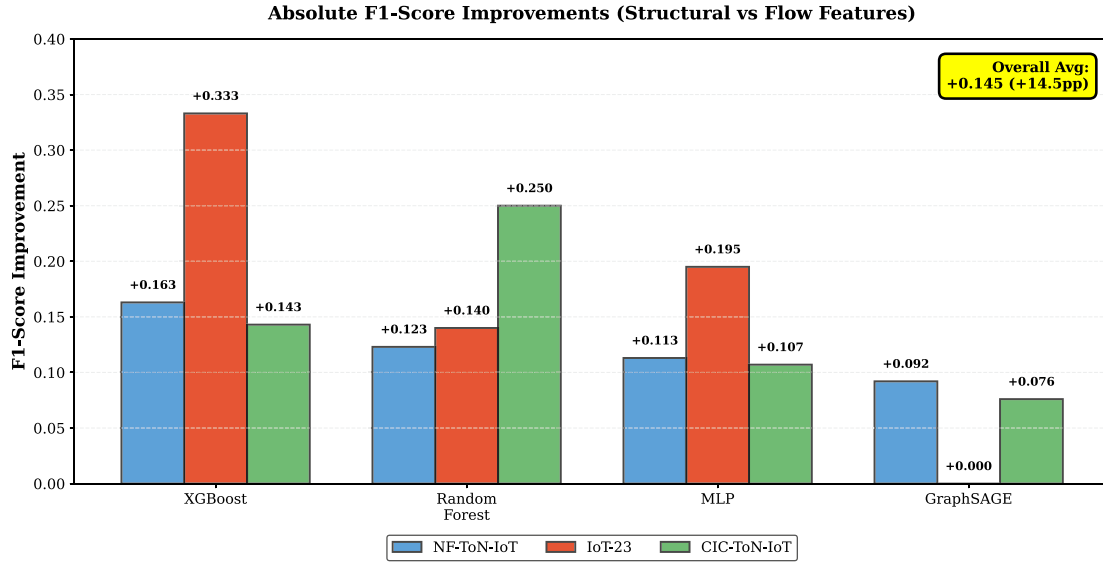


Fig. 6. Absolute F1-score improvements achieved by AdapCVP's structural features over data-flow baselines, showing consistent gains across all classifiers and datasets.

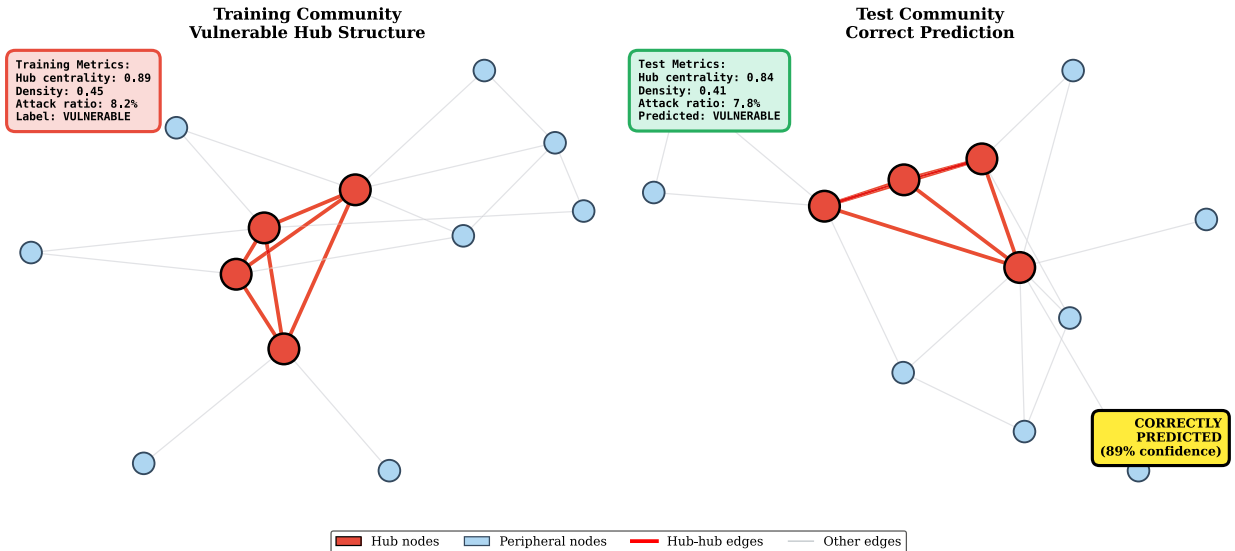


Fig. 7. Structural vulnerability prediction example from NF-ToN-IoT dataset illustrating how the framework learns generalizable vulnerability patterns. Left: Training community exhibiting vulnerable hub structure with high betweenness centrality (0.89), moderate density (0.45), and elevated attack ratio (8.2%), correctly labeled as vulnerable during training. Right: Test community displaying similar structural characteristics with comparable hub centrality (0.84), density (0.41), and attack ratio (7.8%), correctly predicted as vulnerable with 89% confidence despite never being observed during training. The model learns that hub-centric communities with specific density-centrality-attack patterns indicate vulnerability, enabling generalization to unseen communities with analogous structural properties.

Table 8
Impact of Aggregated Data-Flow Traffic Features.

Method	NF-ToN-IoT	IoT-23	CIC-ToN-IoT
<i>Structural Features Only (27 features)</i>			
XGBoost	0.941	1.000	1.000
Random Forest	0.941	0.909	1.000
MLP	0.875	0.909	0.857
GraphSAGE	0.625	0.714	0.933
<i>Structural + Node Aggregates (32 features)</i>			
XGBoost	0.883	0.988	1.000
Random Forest	0.871	0.921	0.986
MLP	0.857	0.921	0.871
GraphSAGE	0.643	0.732	0.914
<i>Average Absolute Difference</i>			
All Models	0.041	0.014	0.012

and how communities interact with the broader network infrastructure. These organizational properties determine vulnerability because attackers exploit structural positions like hub devices controlling information flow or bridge communities connecting isolated network regions. Traffic volume statistics like average packets per flow or byte rate variance indicate communication intensity but provide limited information about organizational structure. A highly vulnerable hub community and a secure peripheral community might exhibit similar average traffic volumes if the hub handles diverse low-intensity flows while the peripheral group exchanges high-volume sensor data. The structural features capturing hub centrality, density, and inter-community connectivity patterns distinguish these cases effectively, making additional traffic statistics redundant. This finding also reinforces the privacy-preserving nature of the approach, demonstrating that effective vulnerability prediction requires only connection topology without needing detailed traffic

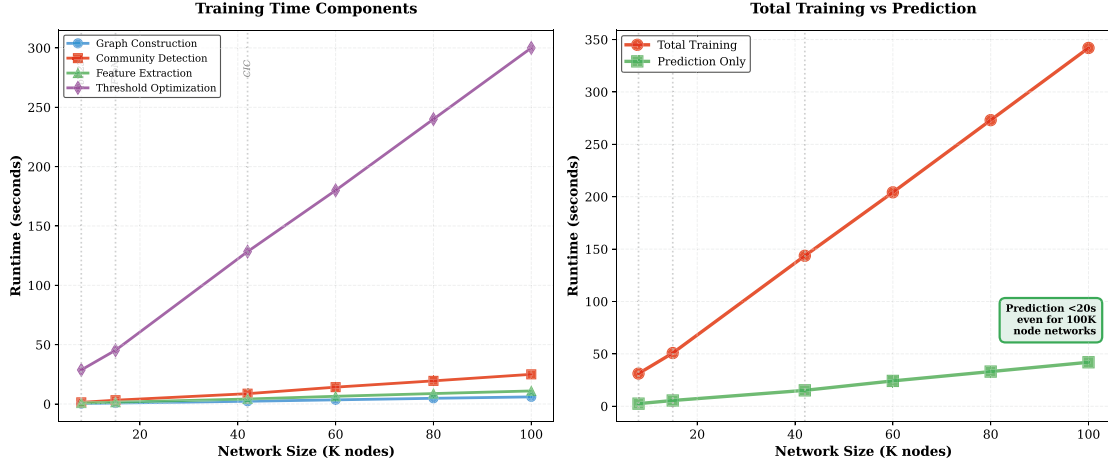


Fig. 8. Computational complexity scaling analysis showing framework runtime as a function of network size. Graph construction and community detection scale efficiently with node and edge count. Feature extraction scales with community count rather than node count, providing efficiency gains. Threshold optimization dominates training time but executes only once during initial deployment.

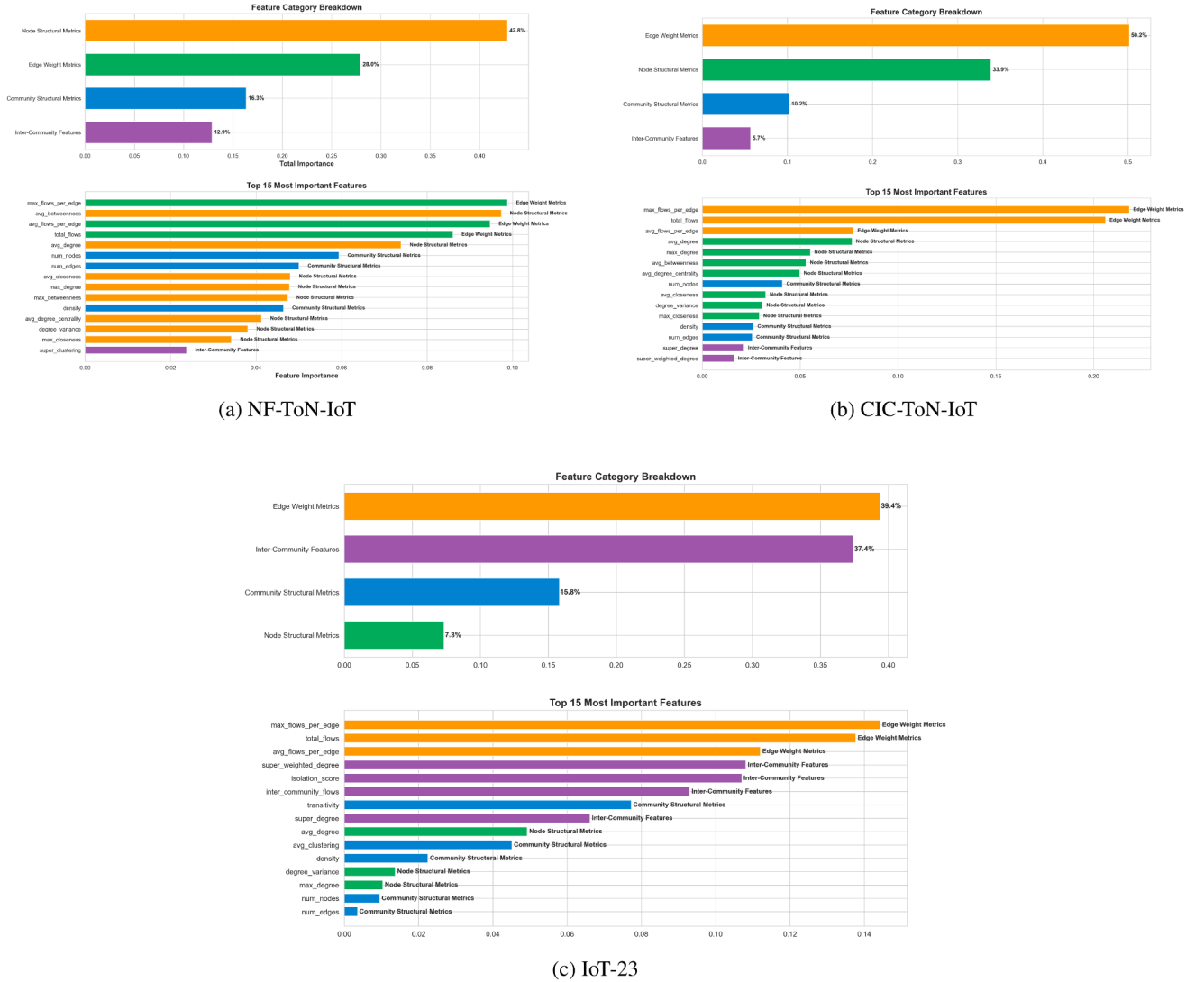


Fig. 9. Feature importance distribution across three IoT datasets. Each panel shows the percentage contribution of feature categories to vulnerability prediction. The relative importance of structural categories varies with topology: centralized graphs emphasize node-level metrics, distributed graphs reflect a more balanced contribution between intra-community structure and inter-community connectivity, and clustered graphs are driven by intra-community communication intensity.

Table 9

Feature importance distribution across the three constructed graphs using 26 structural features (attack flows excluded). Values represent the percentage contribution of each feature category to Random Forest prediction performance.

Feature Category	NF-ToN-IoT	IoT-23	CIC-ToN-IoT
Intra-Community Features (Total)	87.1%	62.5%	94.3%
Node Structural Metrics	42.8%	7.3%	33.9%
Edge Weight Metrics	28.0%	39.4%	50.2%
Community Structural Metrics	16.3%	15.8%	10.2%
Inter-Community Features (Total)	12.9%	37.5%	5.7%
Inter-Community Features	12.9%	37.5%	5.7%
Total	100%	100%	100%

characteristics that might reveal sensitive information about device behavior or user activity.

4.6. Feature importance analysis

To identify which structural properties contribute most to community vulnerability prediction, an ablation study was conducted to analyze feature importance across the 26 pure structural features (excluding attack flows, which would circularly dominate as it directly measures the label). Random Forest importance rankings [16] were used to quantify each feature's predictive contribution across the three constructed graphs. Table 9 and Fig. 9 summarize the resulting distributions. We categorize structural features into four groups where node structural metrics include topology indicators such as degree and centrality measures that capture device-level positioning, edge weight metrics represent the weighted topological structure by quantifying communication intensity within communities (e.g., total or average flows per edge), community structural metrics describe structural properties of the community as a whole, such as internal connectivity patterns and inter-community features reflect cross-community connectivity at the inter-community graph level. Intra-community features account for 87.1%, 62.5%, and 94.3% of total importance in NF-ToN-IoT, IoT-23, and CIC-ToN-IoT, respectively, with the remaining contribution attributed to inter-community structure. At the category level, NF-ToN-IoT is dominated by Node Structural Metrics (42.8%), followed by Edge Weight Metrics (28.0%). IoT-23 shows a different pattern, where Edge Weight Metrics (39.4%) and Inter-Community Features (37.5%) jointly provide most of the predictive signal. In CIC-ToN-IoT, Edge Weight Metrics clearly lead (50.2%), with Node Structural Metrics forming a strong secondary component (33.9%). These shifts indicate that the relative importance of structural signals is not fixed, but changes depending on the organization of the underlying graph.

These importance patterns align directly with the structural properties of each graph. NF-ToN-IoT exhibits high centralization (0.732), meaning that a small number of nodes occupy structurally critical positions. This explains why Node Structural Metrics are dominant: in a hub-oriented architecture, vulnerability is strongly associated with

nodes that control many communication paths. Edge Weight Metrics remain relevant, but structural position is more informative than volume alone, since high traffic naturally concentrates at hubs. In contrast, IoT-23 represents a distributed graph with negligible clustering (0.0008) and no dominant hubs. In such a topology, centrality loses discriminative strength, which explains the low contribution of Node Structural Metrics (7.3%). Instead, vulnerability is reflected in communication intensity and in the connections linking different groups, justifying the nearly equal dominance of Edge Weight and Inter-Community features. When no single node governs the structure, cross-community links and flow concentration become the primary indicators of susceptibility. CIC-ToN-IoT, characterized by high clustering (0.247), forms tightly grouped local structures. In this setting, abnormal communication volume within communities becomes highly distinguishable, which justifies the dominance of Edge Weight Metrics (50.2%). Node Structural Metrics remain important because locally central nodes influence how activity spreads inside clusters, but Inter-Community Features play a minimal role due to limited cross-group interaction. Overall, the results confirm that vulnerability signatures depend on graph organization: centralized graphs emphasize node position, distributed graphs emphasize bridges and flow concentration, and clustered graphs emphasize internal communication intensity.

4.7. Computational complexity analysis

Table 10 presents computational complexity and runtime performance across datasets with varying graph sizes. Graph construction exhibits linear complexity in edges, scaling from small networks to large-scale deployments. Community detection scales at approximately $O(|V| \log |V|)$ for the Louvain algorithm, maintaining efficiency even for networks with tens of thousands of nodes. Feature extraction scales linearly with community count rather than node count, completing within seconds across all datasets. Threshold optimization dominates computational cost with complexity $O(C \cdot K \cdot N)$ where C is candidate thresholds, K is cross-validation folds, and N is classifier training cost. Total training time ranges from seconds for small networks to minutes for large-scale deployments, while prediction time remains under one second across all datasets regardless of network size, as illustrated in Fig. 8.

The multi-algorithm community detection approach provides comprehensive multi-resolution analysis by combining three complementary methods (Louvain with multiple resolution parameters, label propagation, and greedy modularity maximization), ensuring robust identification across diverse network topologies. While this approach completes in 22.3 seconds for the largest evaluated network (49K nodes), deployment scenarios with stricter timing constraints may benefit from optimization strategies. Single-algorithm detection using only Louvain provides faster execution while detecting communities at a single resolution rather than multiple granularities. Parallel execution of the three detection algorithms on multi-core systems can reduce wall-clock time through concurrent processing. Incremental detection that re-analyzes only modified network subgraphs when topology changes are localized offers additional speedup for networks experiencing partial rather

Table 10
Computational Complexity and Runtime Performance Analysis.

Component	Complexity	NF-ToN-IoT (1.5K nodes)	IoT-23 (6.6K nodes)	CIC-ToN-IoT (49K nodes)
Graph Construction	$O(E)$	1.8s	7.7s	232.7s
Multi-Algorithm Detection	$O(V \log V)$	0.5s	1.9s	22.3s
Intra-Community Feature Extraction	$O(C \cdot \bar{C})$	0.8s	1.2s	7.2s
Inter-Community Feature Extraction	$O(C ^2)$	0.2s	0.3s	0.4s
Threshold Optimization	$O(C \cdot K \cdot N)$	12.5s	18.3s	28.9s
Model Training (XGBoost)	$O(T \cdot F \cdot C)$	0.03s	0.04s	0.03s
Total Training Time		15.8s	29.4s	291.5s
Total Prediction Time		0.5s	1.0s	0.11s

than wholesale structural evolution. These optimization strategies enable practitioners to balance comprehensive multi-resolution analysis against specific deployment timing requirements based on their operational context.

5. Conclusion

Internet of Things networks in critical infrastructure face sophisticated attacks that exploit structural vulnerabilities, yet existing intrusion detection approaches analyze individual devices in isolation, missing vulnerability patterns that emerge when functionally related devices organize into communities. This work addresses the challenge of predicting vulnerable communities in heterogeneous IoT networks through multi-scale structural analysis. The AdapCVP framework detects communities using multiple algorithms to capture organizational structure at different resolutions, extracts features describing how devices connect within each community and how communities connect to each other, then employs adaptive threshold optimization to automatically determine vulnerability criteria based on network structural characteristics. By operating exclusively on connection topology without examining packet contents, the approach remains applicable to privacy-sensitive deployments where deep packet inspection is infeasible or prohibited.

Experimental validation on three structurally distinct IoT datasets establishes three key findings. First, community-level analysis provides critical advantages over device-level approaches: capturing coordinated attack patterns that manifest across multiple devices simultaneously, reducing computational complexity by aggregating thousands of devices into manageable communities, and enabling proactive vulnerability detection before attacks spread beyond community boundaries. The two-level structural abstraction combining intra-community organization with inter-community positioning consistently improves vulnerability prediction, with F1-score gains ranging from 12.3 to 16.7% across multiple classifier families, averaging 14.5% overall. Second, adaptive threshold optimization successfully identifies appropriate vulnerability criteria automatically, with selected thresholds varying dramatically from 0.5% for distributed networks where any attack concentration signals compromise, to 2% for centralized networks where hub devices naturally concentrate diverse traffic, and 3% for clustered networks. This automatic adaptation validates that vulnerability stems primarily from organizational structure rather than traffic volume, and eliminates manual parameter tuning across heterogeneous IoT networks. Third, structural vulnerability indicators transfer effectively across heterogeneous topologies, with the negligible impact of aggregated node-level traffic statistics (F1-score changes of only 0.012 to 0.041) confirming that vulnerability stems from structural positions like hub devices controlling information flow rather than communication intensity patterns. Fourth, ablation analysis clarifies the structural drivers of prediction performance. While intra-community organization captures the primary vulnerability signal across datasets, the relative importance of structural categories depends on topology. In the centralized NF-ToN-IoT graph, where traffic concentrates around hub nodes, node-level structural metrics provide the strongest signal. In the clustered CIC-ToN-IoT graph, characterized by tightly connected groups, communication intensity within communities dominates. In the distributed IoT-23 graph, where no single hub governs connectivity, prediction reflects a more balanced contribution of intra-community structure and inter-community connectivity. These results further demonstrate that vulnerability indicators are structurally grounded and topology-dependent, emerging from how communities are internally organized and externally connected rather than from traffic volume alone.

Several directions merit future investigation. First, incorporating temporal dynamics would enable tracking how community vulnerability evolves as network structure changes over time. Second, extending the approach to directed graphs would capture asymmetric communication patterns common in IoT networks. Third, investigating causal relationships between structural properties and vulnerability would provide

deeper insights into which organizational characteristics enable or prevent attack success. Fourth, deploying the framework in federated learning scenarios would enable collaborative vulnerability prediction across multiple organizations without sharing sensitive network topology information. Finally, validating the approach on additional IoT domains would establish the generalizability of structural vulnerability indicators across the full spectrum of IoT application contexts.

The AdapCVP framework provides a foundation for scalable, privacy-preserving security assessment in heterogeneous IoT environments by demonstrating that multi-level structural analysis with adaptive threshold selection enables accurate vulnerability prediction without examining communication content. This capability addresses a critical gap in current intrusion detection research, where existing approaches either require deep packet inspection incompatible with privacy requirements or employ fixed parameters that fail to generalize across diverse network topologies. By extracting vulnerability signals exclusively from connection patterns and automatically adapting detection criteria to network characteristics, the framework offers a practical approach to proactive security assessment suitable for deployment across the wide range of IoT topologies found in modern critical infrastructure.

Additional information

Code availability: Source code is available at: <https://github.com/Fouad-AITfaily/IoT-Community-Vulnerability-Prediction.git>

CRedit authorship contribution statement

Fouad Al Tfaily: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Visualization, Writing – original draft, Writing – review & editing; **Zakariya Ghalmane:** Supervision, Validation, Visualization, Writing – review & editing; **Mohamed El Amine Brahmia:** Project administration, Resources, Software, Supervision, Validation, Writing – review & editing; **Hussein Hazimeh:** Project administration, Supervision, Validation, Writing – review & editing; **Ali Jaber:** Project administration, Supervision, Validation, Writing – review & editing; **Mourad Zghal:** Project administration, Supervision, Validation, Writing – review & editing.

Data availability

<https://github.com/Fouad-AITfaily/IoT-Community-Vulnerability-Prediction>. Public datasets: NF-ToN-IoT, CIC-ToN-IoT, IoT-23

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by Ektidar, a Lebanese project aimed at empowering youth, and CESI EAST Region.

References

- [1] W.A. Al-Awsi, Intrusion detection in IoT networks using machine learning techniques, *Int. J. Comput. Inf. Technol.* 2 (8) (2024) 1–15.
- [2] P. Aravamudhan, T. Kanimozhi, A comprehensive survey of intrusion detection systems using advanced technologies, in: 2022 6th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2022, pp. 1–8.
- [3] S. Fortunato, D. Hric, Community detection in networks: a user guide, *Phys. Rep.* 659 (2016) 1–44.
- [4] P. Holme, B.J. Kim, C.N. Yoon, S.K. Han, Attack vulnerability of complex networks, *Phys. Rev. E* 65 (5) (2002) 056109.

- [5] L. Akoglu, H. Tong, D. Koutra, Graph-based anomaly detection and description: a survey, *Data Min. Knowl. Discov.* 29 (3) (2015) 626–688.
- [6] Z. Ghalmane, M. El Hassouni, C. Cherifi, H. Cherifi, Centrality in modular networks, *EPJ Data Sci.* 8 (1) (2019) 1–27.
- [7] Z. Ghalmane, C. Cherifi, H. Cherifi, M.E. Hassouni, Centrality in complex networks with overlapping community structure, *Sci. Rep.* 9 (1) (2019) 10133.
- [8] Z. Ghalmane, M.-E.-A. Brahmia, M. Zghal, H. Cherifi, A stochastic approach for extracting community-based backbones, in: *International Conference on Complex Networks and Their Applications*, Springer, 2022, pp. 55–67.
- [9] Z. Ghalmane, M. El Hassouni, C. Cherifi, H. Cherifi, K-Truss decomposition for modular centrality, in: *2018 9Th International Symposium on Signal, Image, Video and Communications (ISIVC)*, IEEE, 2018, pp. 241–248.
- [10] Z. Ghalmane, C. Cherifi, H. Cherifi, M. El Hassouni, A community-aware backbone extractor for weighted networks, in: *9Th International Conference on Complex Networks and Their Applications*, 2020, pp. p–3.
- [11] F. Al Tfaily, Z. Ghalmane, M.e.A. Brahmia, H. Hazimeh, A. Jaber, M. Zghal, Graph-based federated learning approach for intrusion detection in IoT networks, *Sci. Rep.* 15 (1) (2025) 41264.
- [12] X. Yuan, N. Zhou, S. Yu, H. Huang, Z. Chen, F. Xia, Higher-order structure based anomaly detection on attributed networks, in: *2020IEEE International Conference on Big Data*, Atlanta, GA, USA, 2020, pp. 2691–2700.
- [13] V.D. Blondel, J.L. Guillaume, R. Lambiotte, E. Lefebvre, Fast unfolding of communities in large networks, *J. Stat. Mech: Theory Exp.* 2008 (10) (2008) P10008.
- [14] U.N. Raghavan, R. Albert, S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, *Phys. Rev. E* 76 (3) (2007) 036106.
- [15] R. Mahmoud, T. Yousuf, F. Aloul, I. Zuolkernan, Internet of things (IoT) security: current status, challenges and prospective measures, in: *2015 10Th International Conference for Internet Technology and Secured Transactions (ICITST)*, London, UK, 2015, pp. 336–341.
- [16] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [17] G. Kumar, H. Alqahtani, Machine learning techniques for intrusion detection systems in SDN: recent advances, challenges and future directions, in: *Computer Modeling in Engineering & Sciences*, 2022.
- [18] M.A. Ferrag, L. Maglaras, S. Moschogiannis, H. Janicke, Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study, *J. Inf. Secur. Appl.* 50 (2020) 102419.
- [19] S. Tsimenidis, T. Lagkas, K. Rantos, Deep learning in IoT intrusion detection, *J. Netw. Syst. Manag.* 30 (1) (2022) 1–40.
- [20] E. Anthi, L. Williams, M. Słowińska, G. Theodorakopoulos, P. Burnap, A supervised intrusion detection system for smart home IoT devices, *IEEE Internet Things J.* 6 (5) (2019) 9042–9053. <https://doi.org/10.1109/JIOT.2019.2926365>
- [21] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, Y. Elovici, N-Balot–network-based detection of IoT botnet attacks using deep autoencoders, *IEEE Pervasive Comput.* 17 (3) (2018) 12–22.
- [22] H. Hamzeh, E. Mugellini, O. Abou Khaled, P. Cudré-Mauroux, Socialmatching + + : a novel approach for interlinking user profiles on social networks, *Proc. PROFILES@ ISWC 2017: Semantic web Conf.*, 22 October 2017, Vienna, Austria (2017).
- [23] H. Hazimeh, E. Mugellini, O.A. Khaled, P. Cudré-Mauroux, Linking user profiles in social networks: a comparative review, *Int. J. Soc. Netw. Min.* 2 (4) (2017) 333–361.
- [24] X. Cheng, W. Zhu, W.Q. Yan, Centrality-Aware collaborative network embedding for overlapping community detection, *IEEE Trans. Network Sci. Eng.* 13 (2026) 2236–2250. <https://doi.org/10.1109/TNSE.2025.3611500>
- [25] T.N. Kipf, M. Welling, Semi-Supervised classification with graph convolutional networks, in: *5Th International Conference on Learning Representations (ICLR)*, Toulon, France, 2017.
- [26] A. Grover, J. Leskovec, Node2vec: scalable feature learning for networks, in: *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 855–864.
- [27] M. Termos, Z. Ghalmane, M.-E.-A. Brahmia, A. Fadlallah, A. Jaber, M. Zghal, Intrusion detection system for IoT based on complex networks and machine learning, in: *2023IEEE Intl Conf on DASC/PiCom/CBDCom/CyberSciTech*, 2023, pp. 1–8. <https://doi.org/10.1109/DASC/PiCom/CBDCom/Cy59711.2023.10361433>
- [28] M. Termos, Z. Ghalmane, M.-E.-A. Brahmia, A. Fadlallah, A. Jaber, M. Zghal, GDLC: A new graph deep learning framework based on centrality measures for intrusion detection in IoT networks, *IoT* 26 (2024) 101214.
- [29] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, H. Wu, Toward early and accurate network intrusion detection using graph embedding, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 5817–5831.
- [30] D.-H. Tran, M. Park, FN-GNN: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems, *Appl. Sci.* 14 (16) (2024) 6932. <https://doi.org/10.3390/app14166932>
- [31] Q. Zhu, S. Rass, P. Schartner, Community-Based security for the internet of things, in: *Security and Privacy in Internet of Things: Challenges and Solutions*, Springer, 2018, pp. 251–273. https://doi.org/10.1007/978-3-319-96145-3_11
- [32] L.F. Herrera-Quintero, A.R. Maigua-Naranjo, G. Bula, A. Reyes, D.M. Giral, An access control for IoT based on network community perception and social trust against sybil attacks, *Int. J. Netw. Manage.* 29 (6) (2019) e2181. <https://doi.org/10.1002/nem.2181>
- [33] Y. Yuan, Y. Huang, Y. Yuan, J. Wang, Dynamic threshold-based two-layer online unsupervised anomaly detector, *CoRR abs/2410.22967* (2024). <https://doi.org/10.48550/ARXIV.2410.22967>
- [34] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: *Icissp*, 2018, pp. 108–116.
- [35] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: bot-lot dataset, *Future Generat. Comput. Syst.* 100 (2019) 779–796. <https://doi.org/10.1016/j.future.2019.05.041>
- [36] F. Al Tfaily, Z. Ghalmane, M. Termos, M.-E.-A. Brahmia, A. Jaber, M. Zghal, Generating realistic cyber security datasets for IoT networks with diverse complex network properties, in: *10Th International Conference on Internet of Things, Big Data and Security*, SCITEPRESS-Science and Technology Publications, 2025, pp. 321–328.
- [37] T.M. Booi, I. Chiscop, E. Meeuwissen, N. Moustafa, F.T. den Hartog, ToN_Iot: the role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion datasets, *IEEE Internet Things J.* 8 (12) (2021) 9919–9929.
- [38] M. Newman, *Networks: An Introduction*, Oxford University Press, 2010. <https://doi.org/10.1093/acprof:oso/9780199206650.001.0001>
- [39] L.C. Freeman, Centrality in social networks conceptual clarification, *Soc. Netw.* 1 (1978) 215–239. <https://api.semanticscholar.org/CorpusID:751590>
- [40] P.J. Mucha, T. Richardson, K. Macon, M.A. Porter, J.-P. Onnela, Community structure in time-Dependent, multiscale, and multiplex networks, *Science* 328 (5980) (2010) 876–878. <https://doi.org/10.1126/science.1184819>
- [41] A. Clauset, M.E.J. Newman, C. Moore, Finding community structure in very large networks, *Phys. Rev. E* 70 (6) (2004) 066111.
- [42] M. Girvan, M.E.J. Newman, Community structure in social and biological networks, *Proc. Natl. Acad. Sci.* 99 (12) (2002) 7821–7826.